

AI Representation Optimizer (Track 5)

Technical Document

1. System Overview

A lightweight system that analyzes Shopify store data and simulates how an AI agent interprets it. The system has three main parts:

1. Data ingestion — Shopify API
2. Analysis layer — LLM + rule-based checks
3. Output layer — diagnostics, suggestions, and before/after comparison

2. Architecture & Data Flow

Flow:

Shopify Store → Backend (Node.js) → Processing Layer → LLM → Output

Components:

Shopify API Layer

- Fetches product data (title, description)

Backend (Node.js / Express)

- Handles API calls
- Preprocesses product data

Processing Layer

- Combines rule-based checks (missing fields, short descriptions) with LLM-based evaluation (semantic understanding)

LLM Layer

- Simulates AI agent interpretation
- Generates issues, improved descriptions, and response comparisons

Output Layer

- Returns issues list, suggestions, and before vs. after responses

3. Key Implementation Decisions

1. Hybrid Approach (Rules + LLM)

Pure LLM reliance was avoided. LLM excels at semantics but is unreliable for deterministic checks. Responsibilities were split:

- Rules — missing fields, length checks
- LLM — quality, ambiguity, and clarity evaluation

2. Simple Backend (No Heavy Infrastructure)

A straightforward Node.js/Express backend was used instead of microservices, given hackathon constraints. This allowed faster iteration and easier debugging.

3. No Database Layer

Data is processed in-memory. No long-term storage is needed, and this significantly reduces system complexity.

4. Controlled Prompts for LLM

Prompts were carefully designed to simulate AI recommendation behaviour and avoid generic or unhelpful outputs.

4. LLM vs. Deterministic Logic

Responsibilities are clearly separated to avoid over-reliance on the LLM.

LLM handles:

- Understanding vague descriptions
- Identifying ambiguity
- Generating improved descriptions
- Simulating AI responses

Deterministic code handles:

- Empty or missing fields
- Length checks
- Basic structure validation

5. Failure Handling

Failure scenarios are explicitly handled:

Case 1 — Shopify API Failure

- Retry once
- If still failing: return error "Unable to fetch store data" and stop gracefully

Case 2 — LLM Returns Invalid Output

- Validate response format
- If invalid: retry once, then fall back to rule-based output only

Case 3 — Unexpected or Empty Product Data

- Skip invalid entries
- Log the issue
- Continue processing remaining products

Case 4 — Timeout or Slow Response

- Set timeout for all LLM calls
- Return partial results instead of blocking the entire response

6. System Degradation Strategy

When components fail, the system degrades gracefully rather than crashing entirely:

- No LLM available — rule-based issues are still surfaced
- Partial data — available fields are still analyzed
- API delay — cached or partial response is returned

7. Known Limitations

- No real-time sync with Shopify
 - Limited to text — no image understanding
 - LLM outputs may vary slightly between runs
 - No ranking based on real conversion data
-

8. What We Would Improve

With more time, the following would be prioritised:

- Structured attribute extraction — auto-detection of size, material, and other fields
- Improved scoring system for issue prioritization
- Real-time store integration
- Image and review analysis
- Feedback loop based on user actions

9. Key Insight from Building

The hardest part was not the API integration. The core challenge was deciding what should be handled by deterministic code versus the LLM.

Overusing the LLM makes the system unstable. Underusing it reduces value. The balance was found by:

- Using rules for structure and deterministic checks
 - Using LLM for interpretation and semantic understanding
-